

Proyecto 10. Lista de comprobación

Criterios para denegar la revisión del proyecto

- Parte de la funcionalidad no ha sido implementada: no hay clase `Card` o `FormValidator`.
- En el proyecto hay preguntas dirigidas a los revisores.

Requisitos del proyecto

General

- El proyecto contiene:
 - `index.html` y `index.css`
 - Un directorio `blocks`
 - Una carpeta `images`
 - Archivos de script `Card.js`, `FormValidator.js`, `index.js` y `utils.js`
 - Un archivo `README.md`
- Las hojas de estilo se conectan en un archivo separado.
- El formato del código es coherente y está indentado siguiendo el mejor método para anidar elementos.
- El archivo `README.md` contiene:
 1. El nombre del proyecto
 2. Una descripción del proyecto y su funcionalidad
 3. Una descripción de las tecnologías y técnicas utilizadas
 4. El enlace a GitHub Pages
- Se utiliza la estructura de archivos BEM.

El proyecto cumple con los siguientes requisitos de estilo del código:

- El proyecto cumple con los principios de la metodología BEM.
- camelCase se utiliza para los nombres de funciones y variables.
- Solo se utilizan sustantivos como nombres de variables.
- Los nombres de las variables describen claramente lo que tienen almacenado en su interior. Si el proyecto tiene varias variables con datos similares, entonces dichas variables tienen nombres únicos pero descriptivos.
- Los sustantivos en plural se utilizan para las colecciones NodeList.
- Para las funciones se utilizan nombres descriptivos que reflejan lo que hacen.
- Los nombres de las funciones comienzan con un verbo.
- Los nombres no deben incluir abreviaturas inapropiadas o confusas.

HTML/CSS

- `normalize.css` se importa en `index.css` antes que otros archivos CSS.
- `viewport` está configurado correctamente, se utilizan `title` y `lang`.

Todas las características enumeradas en el brief se han implementado y funcionan correctamente:

- Se han programado todas las secciones del diseño.
- Se han creado 6 tarjetas a través de JavaScript.
- El formulario para agregar tarjetas ha sido programado correctamente en HTML y CSS, el formulario se puede abrir y al enviarlo se agrega una tarjeta.
- El botón de "Me gusta" funciona.
- La función para eliminar tarjetas se ha implementado correctamente.
- Se puede añadir una tarjeta pulsando Enter mientras un campo de texto está activo.

JavaScript + HTML/CSS

Ventanas modales:

- Las ventanas emergentes (popups) se crean con ayuda de HTML y CSS. No deberían crearse de forma dinámica mediante JS.
- Los cuadros emergentes (popup boxes) pueden cerrarse en cualquier resolución de pantalla.
- El cuadro emergente con la imagen se abre correctamente, y las imágenes se muestran respetando los ratios de aspecto.
- La ventana emergente se puede cerrar al hacer clic en cualquier lugar fuera de sus bordes, o al pulsar la tecla Esc.
- La ventana emergente no se cierra al hacer clic dentro de sus bordes (en el propio formulario, no en la ventana).

Validación:

- Todos los campos de entrada son validados gracias a una función universal.
- Los atributos HTML5 y la propiedad JS `ValidityState` se utilizan para validar los datos de entrada.
- Una función separada controla el estado del botón `Submit`.
- La función `setEventListeners()` es la que sirve para iniciar el proceso de validación.
- El botón `Submit` está inactivo si al menos uno de los campos no pasa la validación.

POO:

- Se utilizan clases de ES6.
- Cada clase (`Card` y `FormValidator`) se describe en un archivo JS separado y se importa en `index.js`.
- Se crea una instancia de la clase `Card` para cada tarjeta. La clase `Card` debe cumplir los siguientes requisitos:
 1. Su método constructor debe tomar los datos de la tarjeta (texto y un enlace a la imagen).
 2. Su método constructor debe tomar el selector de su elemento de plantilla que contiene el código HTML.

3. Tiene métodos privados para añadir detectores de eventos, controlar eventos de clic y preparar la tarjeta para su visualización.
 4. Tiene un método público que devuelve el código HTML terminado con detectores de eventos adjuntos.
- Se crea una instancia de la clase `FormValidator` para cada formulario validado. La clase `FormValidator` debe cumplir los siguientes requisitos:
 - Su método constructor debe tomar un objeto de configuración que almacene los selectores y las clases de formularios.
 - Su método constructor debe tomar una referencia al elemento HTML del formulario que hay que validar.
 - Tiene métodos privados para procesar el formulario. En cada método, debes referirte al campo de la clase, y no pasarlo a cada método, como se implementó anteriormente.
 - Tiene un método público `setEventListeners()`. Llámalo después de crear una instancia de la clase.
 - Cada clase realiza una única tarea. Todo lo relacionado con esa tarea debe estar envuelto dentro de su clase.

El código está optimizado:

- Cualquier dato introducido por el usuario no debe ser asignado a la propiedad `innerHTML`.
- No hay código duplicado. Si una línea de código tiene que repetirse, debe escribirse como una función separada.
- Si se declara una variable utilizando `let`, su valor debe cambiar en algún lugar.
- Si el valor de una variable no cambia, debe declararse con `const`.
- Todos los valores numéricos se asignan a las variables. Los valores únicos que no tienen su propia variable se llaman "números mágicos", y se consideran una mala práctica en programación.
- Las operaciones sobre los elementos del DOM se ejecutan antes de que sean insertados en el diseño.

- Una función realiza una única operación, por ejemplo, devuelve el código HTML de la tarjeta.
- Cuando se abre la ventana emergente, se añade un detector de eventos que permite cerrarla pulsando `Esc` ; cuando la ventana se cierra, este detector se elimina.

Accesibilidad de la interfaz

- Todos los enlaces y elementos interactivos tienen un estado `:hover` .
- Todos los elementos `<img>` tienen un atributo `alt` que contiene una descripción en el idioma de la página.